The lux Commit-on-Idle Reconciliation Discipline: a Z Specification

Formal model of the honour / defer / commit-echo state machine
shared by every continuous-edit element

July 2026

1 Introduction

This is the Z model of lux's *commit-on-idle* reconciliation discipline: the honour / defer / commit-echo state machine that every non-atomic mutable element shares. A non-atomic mutable — a *continuous-edit* — element is one whose value the user changes through a sustained gesture rather than a single click: `input_text`, `slider`, `input_number`, and `color_picker`. All four are reconciled through the one shared `ContinuousEditArbiter`, and this specification governs all four unchanged. It does so because it is *data-independent*: its only carrier, `VALUE` (Section "Basic Types"), is a parameter, not a concrete type, and the machine never inspects the internal structure of a value — only whether two values are equal. The carrier has three valid instantiations, one per carried type: the text a `input_text` holds (`VALUE` := `str`), the number a `slider` or an `input_number` holds (`VALUE` := `float`), and the arity-4 RGBA float tuple a `color_picker` holds (`VALUE` := tuple). The two *atomic-selection* kinds — `checkbox`, `combo`, `radio`, and `selectable` — are out of scope (Section 13): an atomic selection commits in one step and honours the Hub every frame, so it has no buffer, no echo-latency window, no arbiter, and nothing to reconcile.

The exposition below is cast in terms of `input_text`, the element the model was first extracted from and the canonical instantiation (`VALUE` := `str`). Nothing in the argument depends on the value being text: read "text" as "value" and the same reasoning holds for the float and tuple carriers. Section "Basic Types" records why the two other carriers add no new interleaving for the model to miss.

An `input_text` carries one Hub-authoritative `value`, replicated to the Display and rendered every frame by an ImGui text widget that keeps its *own* editable buffer. The renderer must reconcile the two. While the user is not editing, the field *honours* the Hub: each frame renders the replicated value, so an agent-driven change appears on the next frame, in the manner of a checkbox. While the user *is* editing, the local buffer is authoritative and a Hub-driven value is *deferred*, so that two edits in flight before a round trip returns cannot clobber the text under the user's fingers. Exactly one `ValueChanged` fires when an edit commits — on blur or Enter — never per keystroke.

The subtlety the design must survive is the *echo-latency window*. The Hub and the Display are separate processes, potentially on separate machines. When the field commits a value, that value travels to the Hub, the Hub installs it, and only then does it come back round as the replicated `value` the Display renders. Until that echo arrives, the replicated value the Display holds is the *pre-echo* one: the value from before the commit. This document calls that returning value the *echo*, and the interval between a commit and its echo the *echo-latency window*.

Two defects of the same class have surfaced across two review rounds, and that recurrence is why the discipline is modelled here rather than reasoned about informally. The first — pipelined edits clobbering live text — motivated the commit-on-idle redesign, whose author held that "the race is gone by construction". A residual race was then found immediately, so the informal argument did not hold. The second (the residual) is the one this document is built around: *a re-focus during the echo-latency window loses the committed value.*

The residual runs as follows. On commit, the renderer drops its buffer and the field returns to honouring the Hub; but the Hub still holds the pre-echo value, so an idle frame renders that pre-echo value, not the value just committed. If the user re-focuses the field during this window, the current renderer begins deferring on the mere act of focus: it snapshots the *displayed* text — which is the stale pre-echo value — as the buffer and sets the editing flag. The arriving echo of the just-committed value is now deferred for the whole focus session, because the field is editing; and subsequent typing commits from

that stale base, overwriting the value that had just committed. On separate machines this is routine, not exotic.

The model isolates the two decisions that, taken together, produce the loss: *when* the field begins deferring (on focus, or only on a real edit), and *what* an idle field honours during the echo-latency window (the raw pre-echo Hub value, or the value it optimistically knows it committed). It proves that the corrected design — defer only after a real edit, and honour the committed value locally until the Hub echo confirms it — admits no losing trace, and that the two tempting partial fixes each still do.

This is a finite state machine over one field and a small fixed set of text values, so ProB can enumerate every interleaving of focus, keystroke, commit, echo, and re-focus, and either confirm the invariants or produce the exact offending trace.

2 Basic Types

$[VALUE]$

`VALUE` is the set of values the field may hold. It is a *given set* — a parameter, not a concrete type — and that is the whole of the model's generality: the machine compares values for equality and never inspects their structure, so the same schemas govern the `str` carrier of an `input_text`, the `float` carrier of a `slider` or an `input_number`, and the RGBA-tuple carrier of a `color_picker` without change. The other two carriers add no new interleaving for the model to miss. The float carrier is a single value crossing the wire exactly as text does; the one type-specific proof obligation — that value equality is reflexive, which fails for a `NaN` — is discharged outside the model by the elements' `validate` guards (`math.isfinite`) and, for the tuple, structurally by its hex-string wire encoding, which cannot represent `NaN`. The tuple carrier is atomic at the wire (one hex string), the widget (one changed/deactivate signal), the commit (one `ValueChanged`), and the echo (one apply), so it exhibits no partial-echo or per-component race absent from the text case.

Two values suffice to exhibit the loss: an initial and pre-echo value, and a distinct committed value whose echo has not yet arrived. Three exercise the interleaving of a second edit against an in-flight echo more thoroughly. The carrier is bounded by ProB's `DEFAULT_SETSIZE`, so no explicit cardinality constant is needed.

3 Free Types and Constants

$ZBOOL ::= ztrue \mid zfalse$

A two-valued flag written as a free type rather than the toolkit `bool`, so ProB animates it as a small enumerated set.

$MAXFIRE == 2$

The fire counter saturates at `MAXFIRE`. One edit session may fire at most once; a count of two is already a witness to a repeated-fire defect, so counting no higher keeps the state space small without hiding a violation.

$\mid\ v0 : VALUE$

`v0` is the value the field starts on — both the initial displayed text and the initial Hub value. A committed value distinct from `v0` is what the echo-latency window is about: after committing it, the Hub still holds `v0` until the echo lands.

4 State

The state is flat: one schema, eleven components. Its predicate carries only *structural* coherence — that a field can defer or be mid-edit only while it is focused, and that the fire counter stays in range. These hold in every design, correct or buggy, so they act as coherence constraints, not as the safety properties under test. The safety properties (Section 9) are deliberately *absent* here, so that a violating state remains reachable and ProB can find it.

Field

$$
\begin{aligned}
&\mathit{disp} : \mathit{VALUE} \\
&\mathit{hub} : \mathit{VALUE} \\
&\mathit{focused} : \mathit{ZBOOL} \\
&\mathit{editing} : \mathit{ZBOOL} \\
&\mathit{edited} : \mathit{ZBOOL} \\
&\mathit{pending} : \mathit{ZBOOL} \\
&\mathit{committed} : \mathit{VALUE} \\
&\mathit{fires} : 0 \mathinner{..} \mathit{MAXFIRE} \\
&\mathit{lost} : \mathit{ZBOOL} \\
&\mathit{clobbered} : \mathit{ZBOOL} \\
\hline
&\mathit{editing} = \mathit{ztrue} \Rightarrow \mathit{focused} = \mathit{ztrue} \\
&\mathit{edited} = \mathit{ztrue} \Rightarrow \mathit{focused} = \mathit{ztrue}
\end{aligned}
$$

`disp` is the text ImGui currently renders — the value a commit would fire. `hub` is the Hub-authoritative replicated value. `focused` records whether the widget is being interacted with. `editing` is the *defer* flag: while it is set the buffer is authoritative and the Hub value is ignored. `edited` is the *real-edit* witness: whether a genuine keystroke has changed the text since the current focus began; it distinguishes a field the user is merely focused on from one being edited, and it is the component the honour property is stated over.

`pending` is the echo-latency flag: set from a commit until the echo lands, it marks the window in which the Hub has not yet caught up to the committed value. `committed` is the value most recently committed — the value the field optimistically displays while `pending`, and the value whose durability is under test. `fires` counts commit fires since the current focus began, the window in which at most one fire is legitimate.

The last two components are pure bookkeeping for the invariant checks, not part of the renderer's own state. `lost` latches `ztrue` the moment an edit session begins from a stale pre-echo base — the durability failure. `clobbered` latches `ztrue` the moment an echo overwrites live post-edit text — the no-clobber failure.

5 Initialization

Init

$$
\begin{aligned}
&\mathit{Field}' \\
\hline
&\mathit{disp}' = v0 \\
&\mathit{hub}' = v0 \\
&\mathit{focused}' = \mathit{zfalse} \\
&\mathit{editing}' = \mathit{zfalse} \\
&\mathit{edited}' = \mathit{zfalse} \\
&\mathit{pending}' = \mathit{zfalse} \\
&\mathit{committed}' = v0 \\
&\mathit{fires}' = 0 \\
&\mathit{lost}' = \mathit{zfalse} \\
&\mathit{clobbered}' = \mathit{zfalse}
\end{aligned}
$$

A single initial state: the field unfocused and not editing, display and Hub both on `v0`, no commit outstanding, nothing fired, neither failure latched.

6 The Focus Session

A focus session runs from the user focusing the field to the field losing focus. Focusing begins a fresh session: it clears the real-edit witness and the fire count, and — this is the first half of the corrected design — it does *not* begin deferring. A focused field that has not yet been edited still honours the Hub (Section 8), so an echo arriving mid-focus can still reach the display. This is what distinguishes the corrected design from the current one, whose focus begins deferring at once (Section 11, defect (i)).

Focus

$\Delta Field$

$focused = zfalse$
$focused' = ztrue$
$editing' = zfalse$
$edited' = zfalse$
$fires' = 0$
$disp' = disp$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$lost' = lost$
$clobbered' = clobbered$

A keystroke changes the text. It is the second half of the corrected design: deferring begins here, on the *first* real edit of the session, not on focus. The new text `e?` models whatever the user typed. The durability latch reads the state at the instant deferring begins — the transition `editing` `zfalse` → `ztrue` — and fires if that base is stale: a commit is still in flight (`pending`), yet the base the edit builds on is not the committed value. Under the corrected honour rule this can never happen (Section 9); the latch is here so that the fidelity variants, which weaken that rule, have somewhere to trip it.

Keystroke

$\Delta Field$
$e? : VALUE$

$focused = ztrue$
$lost' =$
  if $editing = zfalse \land pending = ztrue \land disp \neq committed$
  then $ztrue$ **else** $lost$
$editing' = ztrue$
$edited' = ztrue$
$disp' = e?$
$focused' = focused$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$fires' = fires$
$clobbered' = clobbered$

A commit is the deactivation of a field that was being edited — blur or Enter after a real edit. It fires exactly once, records the committed value, and opens the echo-latency window (`pending`). The Hub is *not* updated here: the echo is delayed, and arrives only later, in `HubEcho`. The display keeps the committed value: because `pending` is now set, the following honour frame renders `committed`, not the stale Hub value.

Commit

$\Delta Field$

$focused = ztrue$
$editing = ztrue$
$committed' = disp$
$pending' = ztrue$
$fires' = \textbf{if}\ fires < MAXFIRE\ \textbf{then}\ fires + 1\ \textbf{else}\ MAXFIRE$
$focused' = zfalse$
$editing' = zfalse$
$edited' = zfalse$
$disp' = disp$
$hub' = hub$
$lost' = lost$
$clobbered' = clobbered$

A field may also lose focus without ever having been edited: the user focused it and moved away. This deactivation fires nothing — there is no edit to commit — and is the reason a fire count is per session rather than global.

BlurNoEdit

$\Delta Field$

$focused = ztrue$
$editing = zfalse$
$focused' = zfalse$
$edited' = zfalse$
$editing' = zfalse$
$disp' = disp$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$fires' = fires$
$lost' = lost$
$clobbered' = clobbered$

7 The Hub Value

Two operations move the Hub value. `HubEcho` is the delayed echo of a commit: the committed value finally arrives at the Hub and the echo-latency window closes. From here on an honour frame renders `hub`, which now equals `committed`, so the optimistic display and the real Hub value have converged.

HubEcho

$\Delta Field$

$pending = ztrue$
$hub' = committed$
$pending' = zfalse$
$disp' = disp$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$committed' = committed$
$fires' = fires$
$lost' = lost$
$clobbered' = clobbered$

`AgentPush` is an agent-driven change to the Hub value while the field is settled — idle, not editing, no commit outstanding. It is the checkbox-style honour case: a value pushed from outside must appear on

the next honour frame. It is guarded to the settled state so that it exercises honour without entangling with the echo-latency window.

AgentPush

$\Delta Field$
$w? : VALUE$

$focused = zfalse$
$editing = zfalse$
$pending = zfalse$
$hub' = w?$
$disp' = disp$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$pending' = pending$
$committed' = committed$
$fires' = fires$
$lost' = lost$
$clobbered' = clobbered$

8 The Render Frames

Each frame in which the field is *not* deferring honours: it drives the display to the value the field is currently authoritative for. That value is the committed value while an echo is in flight (`pending`), and the raw Hub value otherwise. Honouring the committed value during the window is the optimistic local echo that closes the durability defect: the display shows what the user just committed, so a re-focus that lands in the window — and any edit begun from it — builds on the committed value, not on a stale pre-echo one.

There are two honour frames, one idle and one focused, distinguished only by `focused`. They share the same honour expression. The focused one is the frame the corrected design adds: it is enabled precisely because focus no longer begins deferring, so a focused-but-not-yet-edited field still tracks the Hub. Deleting it, and making focus defer, is defect (i).

IdleHonour

$\Delta Field$

$focused = zfalse$
$editing = zfalse$
$disp' = \textbf{if } pending = ztrue \textbf{ then } committed \textbf{ else } hub$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$fires' = fires$
$lost' = lost$
$clobbered' = clobbered$

FocusedHonour

$\Delta Field$

$focused = ztrue$
$editing = zfalse$
$disp' = \textbf{if } pending = ztrue \textbf{ then } committed \textbf{ else } hub$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$fires' = fires$
$lost' = lost$
$clobbered' = clobbered$

A deferring frame — `editing` set — writes nothing to the display from the Hub: the buffer is authoritative, so `disp` changes only by a further keystroke. There is no separate schema for it; it is the absence of an honour frame while `editing` holds. That absence is the whole of the no-clobber guarantee, and its removal — an echo written to `disp` regardless of `editing` — is defect (iii).

One of the frames or the session operations is always enabled: an unfocused non-deferring field can `IdleHonour`; a focused non-deferring field can `FocusedHonour`; a deferring field is focused and can `Keystroke` or `Commit`; and `HubEcho` is enabled whenever a commit is in flight. A step is therefore always available, which is why the machine cannot deadlock (Section 9, invariant 5).

9 Invariants

Five properties must hold across all reachable states. They are stated here as predicates over `Field`; how each is discharged is given in Section 10. None is placed in the `Field` schema: a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant 1 — commit durability. No edit session begins from a stale pre-echo base, so a committed value is never silently overwritten by a re-focus and edit that started before its echo arrived. The bad event — deferring begun while a commit is in flight and the base is not the committed value — latches `lost`; the invariant is that it never does:

$$lost = zfalse.$$

Invariant 2 — honour discipline. A field defers only after a genuine edit; it never defers on mere focus. While it has not been edited it honours the Hub, so an idle or focused-but-not-edited field reflects the latest Hub value rather than a frozen snapshot:

$$editing = ztrue \Rightarrow edited = ztrue.$$

The display-level reading — a not-editing field's `disp` tracks the value it is authoritative for — is a corollary: whenever `editing` is clear an honour frame is enabled and drives `disp` to that value, so no not-editing field is left frozen. An `AgentPush` while idle is honoured on the next `IdleHonour`; an echo arriving at a focused-not-edited field is honoured on the next `FocusedHonour`.

Invariant 3 — no clobber. An echo never overwrites live post-edit text. A deferring field's display is moved only by a keystroke, never by a Hub value. The bad event — an echo written to `disp` while editing — latches `clobbered`; the invariant is that it never does:

$$clobbered = zfalse.$$

This is established by construction: `HubEcho` frames `disp` unchanged, and the only writers of `disp` are the two honour frames, both guarded on `editing = zfalse`, and `Keystroke`.

Invariant 4 — commit once. A single focus session fires at most once. With the counter reset on every focus, and a commit both requiring `editing` and clearing `focused`:

$$\mathit{fires} \leq 1.$$

Invariant 5 — deadlock freedom. No reachable state leaves the machine unable to progress. The honour frames partition the not-deferring case by `focused`; a deferring field can always `Keystroke` or `Commit`; so a step is enabled in every state.

10 Verification

Type-checking is a `fuzz` pass:

```
fuzz -t docs/commit_on_idle_reconciliation.tex
```

Invariants 1, 2, 3, and 4 are state properties, checked by asking ProB whether their negation is *reachable*. A goal that is not found means the invariant holds on every reachable state:

```
# Invariant 1 -- commit durability (must report: goal NOT found)
probcli docs/commit_on_idle_reconciliation.tex -model_check -nodead \
  -goal "lost=ztrue" -p DEFAULT_SETSIZE 2

# Invariant 2 -- honour discipline (must report: goal NOT found)
probcli docs/commit_on_idle_reconciliation.tex -model_check -nodead \
  -goal "editing=ztrue & edited=zfalse" -p DEFAULT_SETSIZE 2

# Invariant 3 -- no clobber (must report: goal NOT found)
probcli docs/commit_on_idle_reconciliation.tex -model_check -nodead \
  -goal "clobbered=ztrue" -p DEFAULT_SETSIZE 2

# Invariant 4 -- commit once (must report: goal NOT found)
probcli docs/commit_on_idle_reconciliation.tex -model_check -nodead \
  -goal "fires>1" -p DEFAULT_SETSIZE 2
```

Invariant 5 is the deadlock check over the full reachable state space:

```
# Invariant 5 -- deadlock freedom (must report: no deadlock, no violation)
probcli docs/commit_on_idle_reconciliation.tex -model_check -p DEFAULT_SETSIZE 2
```

All five return the same verdict at `DEFAULT_SETSIZE 3`: the extra value widens the interleaving of a second edit against an in-flight echo without adding a new reachable violation.

11 Fidelity: reproducing the defects

A model that cannot reproduce the known defects proves nothing. Each defect is the corrected specification with one part weakened; each then makes the corresponding invariant's negation reachable. Below, each variant names the edit, the goal that changes from *not found* to *found*, and the shortest offending trace. Under `DEFAULT_SETSIZE 2`, `VALUE = {VALUE1, VALUE2}` with `v0 = VALUE1`; write $v_0 = v0$ and v_1 for the other value.

Defect (i) — defer on focus (the reported residual). This is the current renderer. Two edits, both away from the corrected design: `Focus` sets `editing' = ztrue` (it snapshots the displayed text and begins deferring on the mere act of focus, and carries the same durability latch as `Keystroke`, now on the `editing` `zfalse` → `ztrue` transition it introduces); and both honour frames drop the optimistic echo, writing `disp' = hub` unconditionally (the renderer's `release` dropping the buffer, so an idle frame renders the raw pre-echo Hub value). `FocusedHonour` is thereby dead — a focused field always defers — which is the point. Goals found: invariant 2 (`editing=ztrue & edited=zfalse`) and invariant 1 (`lost=ztrue`). Trace for the honour failure:

1. `Init`: `disp = hub =` v_0, not focused, not editing.

2. `Focus` in the buggy form: `focused = ztrue`, `editing = ztrue`, `edited = zfalse`. This state alone violates invariant 2: the field defers although no edit has occurred.

Trace for the durability failure:

1. `Init`; `Focus`; `Keystroke(`v_1`)`: `disp =` v_1, `editing = ztrue`.
2. `Commit`: `committed =` v_1, `pending = ztrue`, `hub =` v_0 (echo delayed), `disp =` v_1, `focused = zfalse`, `fires = 1`.
3. `IdleHonour` in the buggy form: `disp' = hub =` v_0 — the display reverts to the stale pre-echo value.
4. `Focus` in the buggy form: `editing` goes `zfalse` → `ztrue` while `pending = ztrue` and `disp =` $v_0 \neq v_1$ `= committed` — deferring begins from a stale base and `lost` latches. The arriving echo is now deferred for the whole session, and the next commit will fire v_0, overwriting the committed v_1.

Under the corrected design, `Focus` leaves `editing = zfalse`, and the honour frames keep `disp =` v_1 while `pending`, so no base is ever stale. ProB's own shortest witness for `lost` takes a different route to the same stale base: an `AgentPush` diverges `hub` from `disp`, and a commit fires with no prior keystroke — itself possible only because defer-on-focus lets a field be committed without ever being edited — after which a raw-hub honour and a re-focus defer from the stale value. The loss is thus a property of the defer-on-focus/raw-hub pair, not of one path to it.

Defect (ii) — defer after first edit, but honour the raw Hub (the tempting incomplete fix). This is the naive candidate: gate deferring on a real edit — so invariant 2 is restored — but leave the honour frames writing `disp' = hub` unconditionally, still dropping the optimistic echo. It is the fix an informal argument reaches for, and the model shows it is not enough. Edit: only the two honour frames' `disp'` become `hub`; `Focus` and `Keystroke` are as corrected. Goal found: invariant 1 (`lost=ztrue`); invariant 2 (`editing=ztrue & edited=zfalse`) remains *not found*. Trace:

1. `Init`; `Focus`; `Keystroke(`v_1`)`; `Commit`: `committed =` v_1, `pending = ztrue`, `hub =` v_0, `disp =` v_1, `focused = zfalse`.
2. `IdleHonour` in the buggy form: `disp' = hub =` v_0 — the field honours the raw pre-echo value and the display goes stale, exactly as before, because a not-yet-arrived echo cannot be honoured.
3. `Focus`: `focused = ztrue`, still `editing = zfalse` (correct).
4. `Keystroke(`v_0`)`: the user types before the echo arrives. `editing` goes `zfalse` → `ztrue` while `pending = ztrue` and `disp =` $v_0 \neq v_1$ `= committed` — the base is stale and `lost` latches. The committed v_1 is overwritten by an edit begun from v_0.

This is the model's central finding. Deferring after the first edit closes the *focus-and-sit* route — a field that sits focused honours the echo when it arrives — but it does not close the *type-before-echo* route: a user faster than the round trip still edits from the stale displayed value. The durability defect is not a property of *when* deferring begins but of *what an idle field displays during the window*. Only honouring the committed value locally (the corrected `IdleHonour`/`FocusedHonour`) removes the stale base entirely, and the reachability search confirms it: `lost=ztrue` is *not found* for the corrected design and *found*, via the trace above, for this variant. ProB's shortest witness reaches it through `FocusedHonour` rather than `IdleHonour` — the focused-but-not-edited frame itself pulls `disp` to the stale `hub`, then the first keystroke defers from it — confirming the loss is independent of whether the stale honour happened while idle or while focused.

Defect (iii) — echo over live text (the original clobber). This is the first defect of the two, the one the commit-on-idle redesign was meant to prevent: an echo of an earlier edit arriving while a later edit is live. Edit: `HubEcho` additionally writes `disp' = committed` and, when it does so while `editing` holds, latches `clobbered` — modelling an echo applied to the buffer without regard to whether the field is editing. Goal found: invariant 3 (`clobbered=ztrue`). Trace:

1. `Init`; `Focus`; `Keystroke(`v_1`)`; `Commit`: `committed =` v_1, `pending = ztrue`, `hub =` v_0.

2. `Focus`; `Keystroke(`v_2`)`: a second edit is now live — `editing = ztrue`, `disp =` v_2 — while the first commit's echo is still in flight.

3. `HubEcho` in the buggy form: `disp' = committed =` v_1 while `editing = ztrue` — the echo of the first edit overwrites the live second edit and `clobbered` latches.

Under the corrected design, `HubEcho` frames `disp` unchanged: a deferring field's display is the buffer, moved only by a keystroke, so the echo updates `hub` silently and the live text stands.

Summary. Each defect's weakening makes its invariant's negation reachable; with the corrected design intact, the four reachability goals (`lost=ztrue`, `editing=ztrue & edited=zfalse`, `clobbered=ztrue`, and `fires>1`) all return *not found* and the deadlock check reports none. The difference is the evidence that the model detects the defects the code must prevent — and, for defect (ii), that deferring after the first edit is *not* the fix on its own: the load-bearing change is honouring the committed value locally through the echo-latency window.

12 From the Model to the Code

The corrected design is two changes to the arbiter and its renderer, matching the two corrected halves of the model.

The first is *defer after the first edit*, not on focus. The model's `Focus` leaves `editing = zfalse`; only `Keystroke` sets it. In the renderer, the buffer is snapshotted and the editing flag set only when ImGui reports a genuine edit this frame (its `changed` return, or `is_item_edited`), not merely when `is_item_active` — which is true on mere focus. The arbiter's `keep` becomes conditional: it records the buffer and sets editing when an edit occurred or the field is already editing, and otherwise leaves the field honouring. This restores invariant 2.

The second is *honour the committed value through the echo-latency window.* The model's honour frames render `committed` while `pending`, not the raw Hub value. In the arbiter, a commit no longer drops the buffer to the raw Hub value: it records the committed value and the Hub value observed at commit time, and while not editing `resolve` returns the committed value until the Hub value moves off that observed value — the echo landing, or an agent override — at which point it clears the record and returns the Hub value. This is the model's `pending`/`committed` pair and its `HubEcho` clearing `pending`, and it is the change that removes the stale base entirely, closing invariant 1 including the type-before-echo route that defer-after-first-edit alone leaves open.

Concretely, in `ContinuousEditArbiter` (`continuous_edit_selection.py`):

- Add two slots beside the editing flag, under the element id with fresh suffixes (a committed-value slot and a commit-time Hub-value slot), and clear them in `WidgetState.discard_for` alongside the editing flag so a removed field starts clean. They survive a re-push, like the editing flag, because a commit may still be in flight across the resend.

- Replace `keep(text)` with an edit-gated form, `observe(edited, text)`: when `edited` or the field is already editing, set the editing flag and record `text` as the buffer; otherwise do nothing, leaving the field honouring.

- Add `commit(text, hub_value)`: record `text` as the committed value and `hub_value` as the commit-time Hub value. Do not clear the editing flag here; the deactivation path does.

- Change `resolve(hub_value)`: while editing, return the buffer (unchanged); otherwise, if a committed value is recorded and `hub_value` still equals the recorded commit-time Hub value, return the committed value; otherwise clear the recorded slots and return `hub_value`.

- `release` clears the editing flag and drops the buffer on a non-editing frame, but must *not* clear the committed-value slots — they are cleared only when `resolve` observes the Hub value has moved.

In `InputTextRenderer.render` (`display/renderers/input_text_renderer.py`), thread the widget's `changed` return into the arbiter and record the commit:

- Capture the `changed` flag from `input_text_with_hint`.

- On `is_item_active`, call `observe(changed, text)` in place of `keep(text)`; otherwise call `release`.

- On `is_item_deactivated_after_edit`, both `fire` the `ValueChanged` and call `commit(text, elem.value)` so the committed value is honoured locally until its echo returns.

No other honour- or commit-related behaviour changes. In particular the deferring path is untouched: while a real edit is live the buffer remains authoritative and an echo updates only the Hub value, which is invariant 3 standing as it did.

13 Scope and Known Limitations

The reconciliation is by *value equality* alone. A commit carries no echo token or version number, so the arbiter recognises an echo only by the Hub value returning to the value observed at commit time; and one committed / commit-time Hub-value slot pair records only the *latest* commit. Two limitations follow from that single design choice. Both are transient display artefacts, not data loss — the committed value still reaches the Hub and its own echo still lands — and both require an event to fall *within one echo round-trip*, so on localhost, where that window is sub-millisecond, neither is observable in practice.

Two commits within one round-trip. If a field commits twice before the first echo returns, the single slot pair holds only the second commit and the first commit's Hub value. While the first echo is in flight the display can transiently revert to the intermediate authoritative Hub value — a flicker between the two committed values — before the second commit's own echo lands and the display settles. Widening the record to a queue or tagging each commit with a version would remove the flicker at the cost of the single-slot simplicity; the flicker is preferred while the window is sub-millisecond.

Agent drive back to the commit-time value. An agent that drives the Hub value *back to the exact value observed at commit time* during the echo-latency window is indistinguishable, under value-equality reconciliation, from the pending echo of that commit. The push is therefore masked — treated as the awaited echo — and does not re-render until the field next moves off that value. A distinct third value is honoured immediately (the agent-override case of `resolve`); only the coincidence of the *same* value collides.

Relation to the verified model. Both limitations live outside this specification's verified scope by construction. `AgentPush` (Section "The Hub Value") is guarded to the settled state — `focused = zfalse`, `editing = zfalse`, `pending = zfalse` — so the model never drives the Hub value while a commit is in flight, and the single `committed` component admits only one outstanding commit. The transitions that exhibit the two artefacts — a second commit before an echo, and an agent push *during* `pending` — are thus not reachable in the model, and the five invariants make no claim about them. They are documented here, rather than modelled, because closing them would trade the single-slot / value-equality simplicity for machinery the localhost round-trip does not justify.